# SWNA Workshop

Participant Guide

Stochastic White Noise Analysis

2026

## 1. Setup (Google Colab)

### Step 1 — Open the workshop notebook

1. Go to **colab.research.google.com**
2. In the menu bar, click **File** → **Open notebook**
3. Click the **GitHub** tab
4. Type **pnayga** in the search box and press Enter
5. Select the workshop notebook from the list and open it

### Step 2 — Save your own copy

1. In the menu bar: **File** → **Save a copy in Drive**
2. A copy named "*Copy of workshop_notebook.ipynb*" opens in a new tab
3. Rename it if you like: **File** → **Rename**
4. All your work is now saved to *your* Google Drive

> **No Google account?** You can still run the notebook without saving — just click **Run anyway** when prompted. Your progress won't be saved if you close the tab.

### Step 3 — Run the install cell

Run this cell **first** at the top of the notebook:

```
# Install the package
!pip install git+https://github.com/YOUR_REPO/whitenoise.git -q

# Download sample datasets
!wget -q https://raw.githubusercontent.com/YOUR_REPO/main/sample_data/earthquake_ph.csv
!wget -q https://raw.githubusercontent.com/YOUR_REPO/main/sample_data/sunspot_monthly.csv
!wget -q https://raw.githubusercontent.com/YOUR_REPO/main/sample_data/dna_nucleotide.csv
```

**Note:** The facilitator will provide the correct installation link during the session. If running locally (not Colab), use `pip install whitenoise-swna` or install from the folder: `pip install -e .`

## 2. CSV Format

The package reads **2-column CSV files** with this format:

```
time [months], sunspot_number [count]
1, 12.4
2, 15.1
3, 14.8
```

**Rules:**

- **Row 1** is always the header: `name [unit]`
- **Column 1** = independent variable (time, index, position, ...)
- **Column 2** = observable (what you measured)
- Unitless columns: use empty brackets → `flux []`
- Space before bracket is optional: `time[months]` works too

**Blank rows in your CSV?** Clean them first:

```
import pandas as pd
pd.read_csv('raw.csv', skip_blank_lines=True).dropna(how='all').to_csv('clean.csv', index=False)
```

## 3. The Analysis Pipeline

**Step A**

### Import the package

```
import whitenoise as wn
```

**Step B**

### Load your data

```
time, values, meta = wn.read_csv('earthquake_ph.csv')
print(meta)   # shows column names, units, number of points
```

**Step C**

### Inspect the series

```
wn.plot_series(time, values, metadata=meta)
```

**Step D**

### Compute and plot the MSD

```
lags, msd = wn.compute_msd(values)
wn.plot_msd_empirical(lags, msd, metadata=meta)
```

MSD = Mean Square Displacement. The shape of this curve tells you what kind of memory the system has.

**Step E**

### Choose a model and analyze

```
result = wn.analyze('earthquake_ph.csv', model='exponential')
result.summary()
```

This runs the full pipeline and prints the fitted memory parameter μ with confidence intervals and $R^2$.

**Step F**

### Visualize results

```
wn.plot_diagnostics(result)   # 2×2 figure: series, MSD, PDF, parameters
```

**Step G**

## Export your result

```
result.to_csv('my_result.csv')
```

**Step H**

## Compare multiple datasets

```
comp = wn.compare({
    'Earthquakes': 'earthquake_ph.csv',
    'Sunspots':    'sunspot_monthly.csv',
}, model='exponential')

comp.summary_table()        # DataFrame of μ values side by side
wn.publish_comparison(comp) # Bar chart with 95% CI error bars
```

**Step I**

## Batch process a folder

```
batch = wn.batch_analyze('my_data_folder/', model='cosine')
batch.to_csv('batch_results.csv')
```

## 4. The Memory Parameter μ

The central result of each analysis is the **memory parameter μ**, extracted by fitting the MSD curve.

| μ range | Regime | What it means |
|---|---|---|
| μ < 0.95 | Subdiffusive | System resists spreading — fluctuations slow down over time |
| 0.95 – 1.05 | Near-Brownian | Weak or no memory — behaves like an ordinary random walk |
| 1.05 – 2.0 | Superdiffusive | Fluctuations reinforce each other — fast spreading |
| μ > 2.0 | Hyperballistic | Extremely persistent dynamics — very strong memory |

> **fBm model:** Reports the Hurst exponent *H* instead of μ. H = 0.5 → Brownian; H < 0.5 → subdiffusive; H > 0.5 → superdiffusive.

## 5. Available Models

| Model name | Parameters | Best for |
|---|---|---|
| `'exponential'` | μ, β | Decay-type dynamics, inter-event times (earthquakes) |
| `'cosine'` | μ, ν | Oscillatory systems (sunspots, $CO_2$, climate) |
| `'sine'` | μ, ν | Oscillatory — like cosine, wider valid range |
| `'fbm'` | H | Classical fractional Brownian motion |
| `'dna'` | a, b, c | Restricted diffusion (DNA loop distances) |

**How to choose:**

- Start with `'exponential'` — most robust for most datasets
- Check $R^2$: **≥ 0.8** good | **0.5–0.8** moderate | **< 0.5** try another model
- Low $R^2$? Try a different model: re-run `wn.analyze(..., model='cosine')`
- See all models: `wn.list_models()`

## 6. Sample Datasets

| File | Description | Suggested model | Expected μ |
|---|---|---|---|
| `earthquake_ph.csv` | Seismic inter-event times | `exponential` | ≈ 1.00 – 1.19 |
| `sunspot_monthly.csv` | Monthly sunspot counts | `exponential` | ≈ 1.15 |
| `dna_nucleotide.csv` | DNA nucleotide loop separation distances | `dna` | a ≈ 5.2, b ≈ 0.002 |

## 7. Optional Preprocessing

These steps are optional — start without them and add only if needed.

```
# Remove a linear trend (to extract fluctuations)
fluct = wn.detrend(values, method='linear')

# Normalize if data has extreme scale (makes N parameter stable)
scaled = wn.normalize(values, method='zscore')

# Smooth noisy data
smooth = wn.smooth(values, window=7)

# Then analyze preprocessed data
result = wn.analyze(fluct, model='cosine', label='Detrended data')
```

> **When to normalize?** If your fit summary shows `N = 0.0000`, your data amplitude may be very small. Normalize first: `wn.normalize(values, method='zscore')`, then pass the result to `wn.analyze()`. The memory parameter μ is **not affected** by normalization.

## 8. Analyzing Your Own Data

```
# Point analyze() at your CSV file
result = wn.analyze('my_data.csv', model='exponential')
result.summary()
wn.plot_diagnostics(result)
```

Your CSV must have:

- Exactly 2 columns (or more for batch analysis)
- Header row with `name [unit]` format
- No missing values in the middle of the series

If your file is in a subfolder:

```
result = wn.analyze('dataexo/myfile.csv', model='exponential')
```

## 9. Quick Reference

| Function | What it does |
|---|---|
| `wn.read_csv(path)` | Load CSV → `(time, values, meta)` |
| `wn.plot_series(time, values, metadata=meta)` | Plot the raw time series |
| `wn.compute_msd(values)` | Compute MSD → `(lags, msd)` |
| `wn.plot_msd_empirical(lags, msd, metadata=meta)` | Plot MSD scatter |
| `wn.analyze(path, model='...')` | Full pipeline → `AnalysisResult` |
| `result.summary()` | Print fitted parameters and $R^2$ |
| `wn.plot_diagnostics(result)` | 2×2 diagnostic figure |
| `wn.compare({...}, model='...')` | Compare multiple datasets |
| `wn.batch_analyze(folder, model='...')` | Batch process a folder of CSVs |
| `wn.list_models()` | Show all available models |
| `result.to_csv(path)` | Export result to CSV |
| `wn.publish_msd(result, save_path='fig.pdf')` | Publication-quality MSD figure |
| `wn.publish_comparison(comp, save_path='fig.pdf')` | μ comparison bar chart |
| `wn.detrend(values, method='linear')` | Remove trend from series |
| `wn.normalize(values, method='zscore')` | Normalize series |

SWNA Workshop Participant Guide · 2026